

PONTIFICIA UNIVERSIDAD JAVERIANA
Facultad de Ingeniería – Departamento de Ingeniería de Sistemas

DOCUMENTO DE DISEÑO DE SOFTWARE (SDD)

Generación de Escenarios de Movilidad Urbana para la Toma de Decisiones en Localidades de Bogotá

Estándar base: IEEE Std 1016

Samuel Esteban Campos
Erick Salazar Suárez
Santiago Chitiva Contreras
Felipe Andrés Garrido Flores

Director: Ing. Andrés Oswaldo Calderón Romero, PhD.

Bogotá D.C., 13 de octubre de 2026
Versión 2.1

Tabla de Contenidos

1 Introducción 3
1.1 Propósito del documento 3
1.2 Alcance del sistema 3
1.3 Glosario y acrónimos 4
2 Descripción general del sistema 5
2.1 Contexto del proyecto 5
2.2 Objetivos del diseño 5
3 Arquitectura del sistema 6
3.1 Diagrama arquitectónico 6
3.2 Estilo de arquitectura 7
3.3 Flujos globales 8
4 Diseño de datos 9
4.1 Esquema de base de datos 9
4.2 Diccionario de datos 10
4.3 Estrategia de almacenamiento 12
5 Diseño de componentes (diseño detallado) 13
5.1 Desglose de módulos 13
5.2 Lógica y algoritmos 15
6 Diseño de interfaces 18
6.1 Interfaces internas y externas 18
6.2 Interfaces de usuario 21
7 Atributos de calidad 23
7.1 Seguridad 23
7.2 Rendimiento y escalabilidad 23
7.3 Mantenibilidad y pruebas 24
7.4 Reproducibilidad y portabilidad 24
7.5 Observabilidad y usabilidad 25
8 Apéndices y referencias 25
8.1 Pila tecnológica (Tech Stack) 25
8.2 Registros de decisiones (ADRs) 26
8.3 Trazabilidad con el SRS 27
8.4 Limitaciones conocidas del diseño 28
9 Referencias 28

Lista de Figuras

Figura 1 Vista de componentes y conexiones del sistema. 7
Figura 2 Flujo de una simulación parametrizada y franjas de avance (`FRANJAS_ETAPA`). 8
Figura 3 Flujo de preparación y ejecución del simulador 3D comparativo. 9
Figura 4 Modelo de entidades de datos y su relación. 9
Figura 5 Esquema de la pantalla principal del visor. 22
Figura 6 Esquema del simulador 3D comparativo. 22

Lista de Tablas

Tabla 1 Módulos del sistema cubiertos por este documento. 3
Tabla 2 Glosario de términos y acrónimos. 4
Tabla 3 Objetivos de diseño priorizados. 5
Tabla 4 Capas de la arquitectura. 7

Tabla 5 Columnas del TSV de viajes y su uso. 10
Tabla 6 Campos de `estado.json` (dataclass `Corrida`). 10
Tabla 7 Esquema de cada *feature* de `kepler.geojson`. 11
Tabla 8 Entidades de análisis de intersecciones. 11
Tabla 9 Ubicaciones de almacenamiento y su política. 12
Tabla 10 Módulos de `simulation-service` y su responsabilidad. 13
Tabla 11 Componentes del visor y su responsabilidad. 14
Tabla 12 Interfaces externas del sistema. 18
Tabla 13 Endpoints de la API. 19
Tabla 14 Mensajes del WebSocket `/comparacion3d/{id}`. 20
Tabla 15 Estrategias de rendimiento. 23
Tabla 16 Pila tecnológica. 25
Tabla 17 Registro de decisiones de arquitectura. 26
Tabla 18 Trazabilidad entre grupos de requisitos y componentes de diseño. 27

1 Introducción

1.1 Propósito del documento

Este Documento de Diseño de Software (SDD) describe cómo está construido el sistema *Generación de Escenarios de Movilidad Urbana – Usaquén*: la arquitectura, los datos que maneja, el diseño detallado de cada componente y las interfaces que expone. Sigue la estructura de la norma IEEE Std 1016 [1] y complementa a la Especificación de Requisitos (SRS, [2]), que dice *qué* debe hacer el sistema, y al Plan de Gestión (SPMP, [3]), que dice *cómo* se gestiona el proyecto. Aquí se documenta cómo está resuelto técnicamente.

El documento describe el sistema tal como está implementado en la rama de desarrollo (SRS versión 2.1). Donde algo está diseñado pero no construido, o construido con una limitación conocida, se indica de forma explícita (por ejemplo, el transporte público, Sección 8.4).

Está dirigido a:

- **El equipo de desarrollo**, como referencia para mantener y extender el código.
- **El director del proyecto** y **los jurados evaluadores**, como evidencia de las decisiones de diseño y de la justificación de cada una.
- **Futuros mantenedores o investigadores** que quieran reutilizar el pipeline de simulación con otra localidad de Bogotá.

El sistema se compone de dos módulos que se documentan en conjunto:

Tabla 1: Módulos del sistema cubiertos por este documento.

Módulo	Descripción	Directorio
`simulation-service`	Paquete Python instalable (≥ 3.11). Contiene el pipeline ETL que convierte la Encuesta de Movilidad en entradas de SUMO, ejecuta la simulación y exporta los resultados, y una API REST/WebSocket (FastAPI) que lo expone.	`simulation-service/`
`kepler`	Aplicación web de una sola página (Vite + React) que embebe kepler.gl para animar los viajes simulados y ofrece paneles de demanda, intersecciones, semaforización y el simulador 3D comparativo.	`kepler/`

El resto del repositorio (`notebooks/`, `data/`) es un espacio de exploración de datos (CRISP-DM, fases 1 y 2) cuyos productos entran al sistema como insumos versionados; no forma parte del diseño de software y solo se menciona como fuente de datos.

1.2 Alcance del sistema

Dentro del alcance. El sistema:

- Transforma los viajes de la Encuesta de Movilidad de Bogotá 2023 (filtrados a los internos de Usaquén, ventana 07:00–10:00) en demanda multimodal para SUMO, replicando cada viaje según su factor de expansión.
- Permite parametrizar la demanda (presets de cambio modal, conteos por modo, semilla, estilo de conducción y cierres viales) y ejecutar la simulación como trabajos en segundo plano con avance y cancelación.
- Sirve resultados precalculados de los presets para consulta inmediata.
- Visualiza los viajes animados sobre un mapa y los resume por modo.

- Cataloga las intersecciones semaforizadas de la red, muestra sus accesos, volúmenes simulados y programas, y calcula un programa de Webster.
- Prioriza los cruces sin semáforo que justificarían uno nuevo (reglas viales más evidencia de siniestralidad, volumen y entorno).
- Compara en 3D, y en tiempo real, un cruce tal como está contra el mismo cruce con un semáforo propuesto, con telemetría de riesgo del actor seguido y un modo de conducción manual.

Fuera del alcance.

- **Transporte público real**: no hay buses, paradas ni GTFS. Los viajes en transporte público se simulan como caminata (`<person><walk>`).
- **Calibración y validación estadística del modelo** frente a aforos reales (HMD). El sistema produce las simulaciones, no certifica la exactitud del modelo.
- **Programas semafóricos reales**: los programas son los generados por `netconvert` desde OpenStreetMap, no los de la Secretaría Distrital de Movilidad.
- **Autenticación, multiusuario y despliegue en la nube**: es una herramienta local de investigación (Sección 7.1).
- Otras localidades distintas de Usaquén (el diseño es parametrizable por escenario, pero solo existe el escenario `default`).

1.3 Glosario y acrónimos

Tabla 2: Glosario de términos y acrónimos.

Término	Significado
SUMO	*Simulation of Urban MObility*: simulador microscópico de tráfico de código abierto [4]. Se usa a través de `sumo`, `duarouter`, `netconvert`, `cutRoutes.py` y `randomTrips.py`.
TraCI	*Traffic Control Interface*: protocolo para controlar una instancia de SUMO en ejecución (avanzar pasos, consultar y modificar actores).
FCD	*Floating Car Data*: salida de SUMO con la posición de cada actor en cada periodo. Es la fuente de las exportaciones.
TAZ	*Traffic Analysis Zone*: zona de análisis de tráfico; origen y destino de los viajes (derivadas de las UPZ de Usaquén).
UPZ / ZAT	Unidad de Planeamiento Zonal / Zona de Análisis de Transporte de Bogotá.
OD	Matriz origen–destino.
`fexp`	Factor de expansión de la encuesta: cuántas personas representa un viaje observado.
Preset	Regla de cambio modal predefinida (`encuesta`, `mas_vehiculos`, `mas_peatones`, `dia_sin_carro`) que redistribuye los viajes entre modos.
Corrida (*run*)	Ejecución del pipeline con una demanda concreta; sus archivos viven en `runs/<id>/`.
Precalculada	Corrida de un preset con conteos por defecto, guardada en el repositorio para servirla sin simular.
`vType`	Tipo de vehículo de SUMO; aquí hay uno por modo de viaje (`TransportMode`).
`tlLogic`	Programa de un semáforo en SUMO (fases, duraciones, estados por movimiento).

Cruce	Agrupación de nodos de SUMO próximos que forman una intersección real (p. ej. las dos calzadas de una avenida).
Webster	Método de F. V. Webster para el ciclo óptimo de un semáforo [5].
SSM	*Surrogate Safety Measures*: indicadores de riesgo que no esperan a un choque (TTC, DRAC, PET) [6].
TTC / DRAC / PET	Tiempo a la colisión / tasa de desaceleración para evitarla / tiempo posterior a la invasión del cruce.
Ego	Actor (vehículo o peatón) seguido por la cámara en el simulador 3D; se inyecta con el mismo id y ruta en los dos lados.
Lado	Una de las dos instancias de SUMO del simulador 3D: A = sin semáforo, B = con semáforo propuesto.
Kepler / kepler.gl	Biblioteca de visualización geoespacial; su capa *Trip* anima rutas con marca de tiempo.
ETL	*Extract, Transform, Load*: aquí, encuesta → trips → rutas → simulación → GeoJSON.
SDM	Secretaría Distrital de Movilidad de Bogotá (fuente de la encuesta y de los siniestros).
OSM	OpenStreetMap: fuente de la red vial, nombres de calles, POI y huellas de edificios.

2 Descripción general del sistema

2.1 Contexto del proyecto

Los estudios de movilidad urbana en Bogotá parten de la Encuesta de Movilidad [7], un conjunto tabular de viajes. Convertir esa tabla en una simulación microscópica de tráfico exige muchos pasos manuales y frágiles (zonificar, ponderar, rutear, simular, exportar) y deja a quien toma decisiones sin una forma sencilla de preguntar «¿qué pasaría si…?».

El sistema resuelve tres necesidades del proyecto de grado:

1. **Automatizar** la cadena de datos que va de la encuesta a una simulación SUMO reproducible de Usaquén.
2. **Explorar escenarios hipotéticos** (más vehículos, más peatones, día sin carro, cierres viales, otro estilo de conducción) sin tocar archivos XML, y verlos animados en un mapa.
3. **Apoyar decisiones de semaforización**: identificar qué cruces sin semáforo serían candidatos y mostrar, en una vista 3D comparativa, cómo se comportaría el tráfico con y sin un semáforo propuesto. Esta idea se inspira en el proyecto *Semáforos de Barranquilla* (análisis en `docs/propuesta-semaforizacion-3d.md`).

Los requisitos correspondientes están en el SRS agrupados así: F1 ingestión de datos, F2 matrices OD, F3 red y TAZ, F4 archivos de simulación, F5 variación paramétrica, F6 módulo web, F7 validación y reportes, F8 intersecciones y semaforización, F9 simulador 3D comparativo.

2.2 Objetivos del diseño

Las prioridades que guían las decisiones, en orden, son:

Tabla 3: Objetivos de diseño priorizados.

Prioridad	Objetivo	Cómo se refleja en el diseño

1	**Reproducibilidad científica**: la misma entrada debe producir la misma salida.	Semilla explícita, ordenamiento determinista de viajes, reparto por mayor resto (sin azar), `gzip` con `mtime=0`, huella SHA-256 de los insumos en cada precalculada, versiones de dependencias fijadas.
2	**Mantenibilidad y trazabilidad**: un estudiante nuevo debe poder entender y extender cada etapa.	Un módulo por etapa con el mismo patrón `*Config` / `*Result` / `*Stats`; `configuration.py` como única fuente de rutas; nombres en español; 103 pruebas automáticas.
3	**Respuesta fluida con un simulador lento**: una corrida completa tarda 10–15 min.	Trabajos asíncronos con avance y cancelación, presets precalculados servidos comprimidos, cachés de análisis de red, FCD reducido, descarga con barra de progreso.
4	**Verosimilitud sin sobreprometer**: mostrar resultados creíbles y declarar lo que no se modela.	Notas explícitas en la API (programas de OSM, volumen sin `vehroute`), escena 3D con relleno marcado como ilustrativo (`relleno: true`).
5	**Operación local y simple**: sin infraestructura de base de datos ni nube.	Estado en el sistema de archivos, un solo proceso de API, proxy de Vite para el visor.

Restricciones que condicionan el diseño:

- SUMO es un ejecutable externo (paquete `eclipse-sumo`), de modo que el servicio solo orquesta procesos y archivos; no integra SUMO como biblioteca.
- La API aplica un límite de una corrida pesada a la vez (CPU y memoria de SUMO) y de dos comparaciones 3D simultáneas.
- Los datos crudos de la encuesta no están versionados (viven en Google Drive); lo que entra al repositorio es el TSV de viajes internos ya filtrado.
- El navegador debe poder manejar el volumen de la encuesta completa (24.600 viajes, 64 MB de GeoJSON sin comprimir).

3 Arquitectura del sistema

3.1 Diagrama arquitectónico

El sistema es una aplicación cliente–servidor local. El navegador carga el visor (`kepler`); el servidor de desarrollo de Vite reenvía `/api/*` (HTTP y WebSocket) a la API de `simulation-service` en el puerto 8000. La API orquesta herramientas de SUMO como subprocesos y lee y escribe en el sistema de archivos.

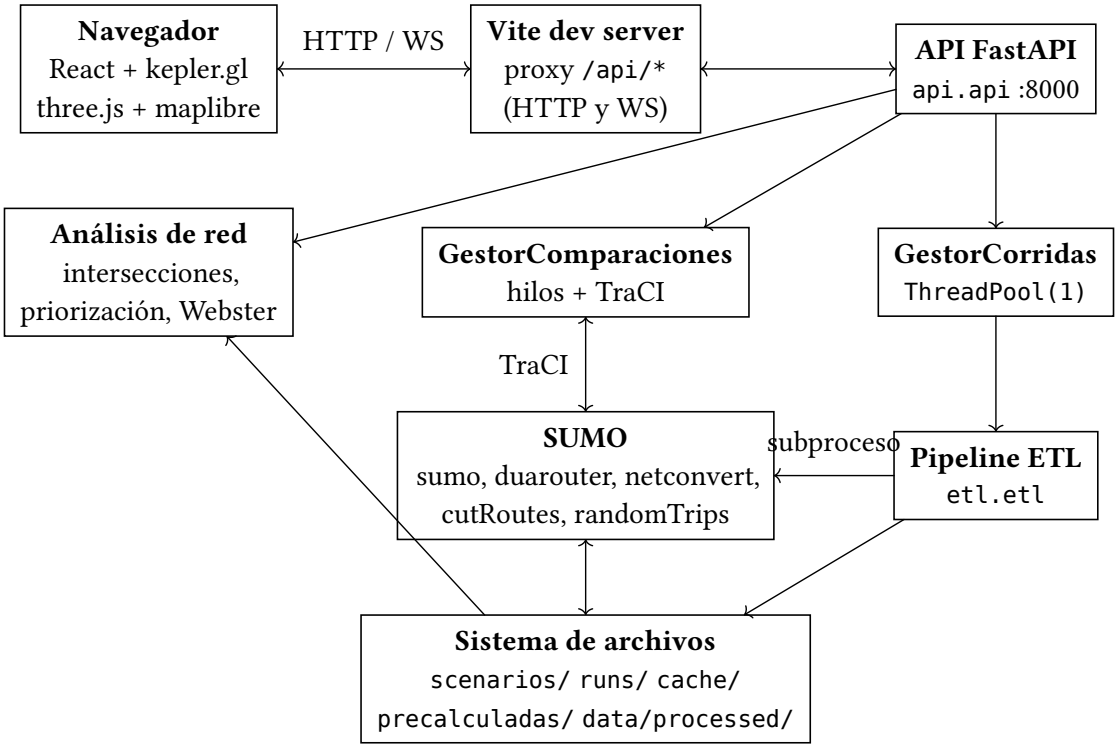


Figura 1: Vista de componentes y conexiones del sistema.

La Figura 1 se organiza en cuatro capas:

Tabla 4: Capas de la arquitectura.

Capa	Responsabilidad
Presentación	`kepler/`: mapa kepler.gl, paneles de demanda, ficha de intersección, semaforización y simulador 3D (carga diferida).
Interfaz (API)	`api/api.py`: validación de peticiones (Pydantic), códigos HTTP, WebSocket. No contiene lógica de negocio pesada.
Orquestación	`api/jobs.py` (corridas en segundo plano) y `api/comparacion3d.py` (comparaciones 3D con TraCI).
Dominio y datos	`etl/`: generación de demanda, ruteo, simulación, exportación, análisis de intersecciones y priorización, escena 3D, telemetría.

3.2 Estilo de arquitectura

Cliente–servidor con servicio monolítico modular. Se eligió un único proceso FastAPI con módulos internos bien separados (API → orquestación → ETL) en lugar de microservicios, porque el sistema se ejecuta en una sola máquina, lo costoso es SUMO y no el enrutamiento, y un solo proceso simplifica la cancelación, el reparto de rutas de archivos y las pruebas (Sección 8.2, ADR-01).

Dentro del servicio conviven tres estilos:

- **Tubería de etapas (*pipes and filters*)** para el ETL: cada etapa lee archivos de la anterior y escribe los propios (TSV → `trips.xml` → `rou.xml` → FCD → GeoJSON). Las etapas son funciones con `Config`/`Result`/`Stats` y se pueden ejecutar solas.
- **Trabajos asíncronos con cola de un solo trabajador** para las corridas largas: la API responde `202 Accepted` y el cliente consulta el estado cada 2 s (*polling*).
- **Flujo en tiempo real por WebSocket** para el simulador 3D: dos instancias de SUMO avanzan en *lockstep* y el servidor emite un *frame* por paso.

Del lado del visor se usa una arquitectura de componentes React con estado local (`useState`) y un *store* Redux solo para kepler.gl, que lo exige. El componente `App.jsx` (`Visor`) es el dueño de los datos del mapa; los paneles reciben propiedades y devuelven eventos.

3.3 Flujos globales

3.3.1 Simulación parametrizada

El usuario ajusta la demanda y lanza una corrida. El flujo atraviesa las cuatro etapas del pipeline, con el avance global repartido en franjas.

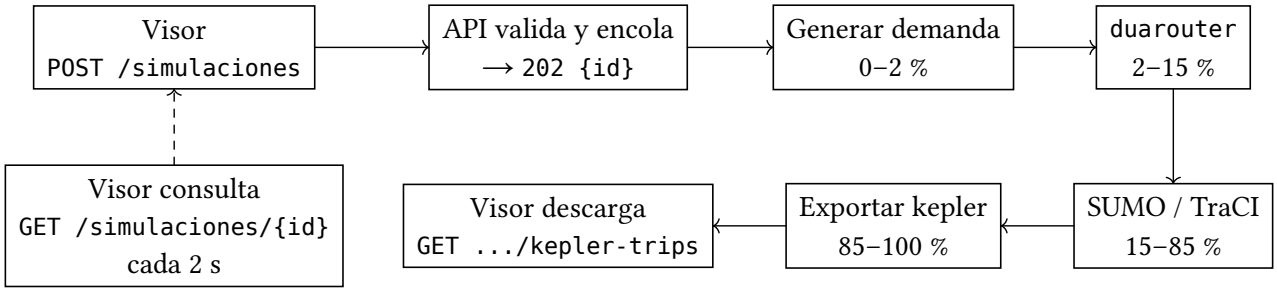


Figura 2: Flujo de una simulación parametrizada y franjas de avance (`FRANJAS_ETAPA`).

1. `POST /simulaciones` valida el cuerpo (total de viajes entre 1 y 100.000) y devuelve `202` con el `id` de la corrida.
2. `GestorCorridas` guarda `runs/<id>/estado.json` y envía el trabajo a un `ThreadPoolExecutor(1)`: solo una corrida pesada corre a la vez y las demás quedan en estado `en_cola`.
3. `run_pipeline` genera los viajes (ordenados por hora de salida), llama a `duarouter`, simula con SUMO bajo TraCI y exporta el GeoJSON de kepler.
4. En cada etapa se informa el avance, que se mapea a la franja global. Una cancelación (`threading.Event`) llega hasta el subproceso y borra todos los archivos de la corrida menos `estado.json`.
5. Al terminar, el FCD (muy pesado) se elimina; el visor descarga los viajes y los reemplaza en kepler.

3.3.2 Simulación precalculada

Los cuatro presets con conteos por defecto se simulan una vez (`python -m etl.precalculadas`) y se versionan en `simulation-service/precalculadas/<preset>/`. El visor, al abrir, consulta `GET /precalculadas`; si el panel de demanda coincide exactamente con una, ofrece «Cargar simulación» y la API entrega el `.gz` tal cual con `Content-Encoding: gzip`, de modo que el navegador descomprime y no hay que simular.

3.3.3 Análisis de intersecciones y priorización

`GET /intersecciones` y `GET /intersecciones/priorizacion` combinan un análisis de la red que no depende de la corrida (catálogo de semáforos, agrupación de cruces, evidencia), guardado en `cache/`, con los volúmenes de la corrida elegida. Los cálculos que dependen de la corrida (puntaje y niveles) se rehacen en cada consulta porque son baratos.

3.3.4 Comparación 3D de un cruce

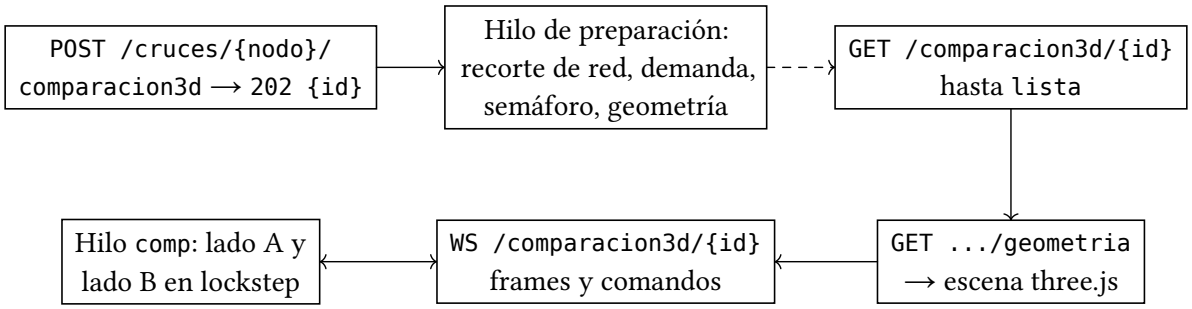


Figura 3: Flujo de preparación y ejecución del simulador 3D comparativo.

La preparación (`netconvert`, `cutRoutes.py`, `randomTrips.py`) corre en un hilo aparte y puede tardar decenas de segundos. Al conectarse el WebSocket se arranca un segundo hilo que lanza los dos SUMO, los calienta 240 s simulados y emite un *frame* por paso (0,2 s simulados). Al cerrar la ventana, el directorio `runs/sem-<id>/` se borra; si nadie se conecta en 180 s, la comparación expira.

4 Diseño de datos

4.1 Esquema de base de datos

El sistema **no usa un motor de base de datos**. Los datos son archivos (TSV, XML de SUMO, JSON y GeoJSON) organizados por escenario y por corrida, y las estructuras en memoria son `dataclass` de Python. Se decidió así porque el volumen es pequeño (un único escenario), los resultados de SUMO ya son archivos, y se evita instalar y operar un servicio adicional (Sección 8.2, ADR-03). El «esquema» se describe, entonces, como el modelo de entidades y su ubicación en disco.

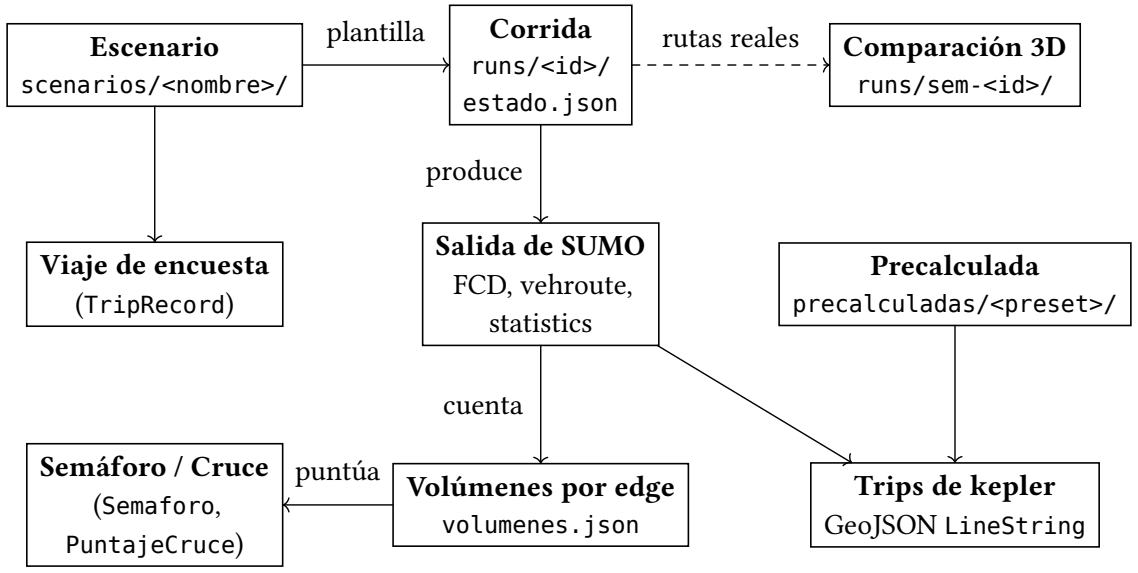


Figura 4: Modelo de entidades de datos y su relación.

La entrada de todo el sistema es el **TSV de viajes internos** (`scenarios/default/viajes_usaquen_internos.tsv`, 2.580 filas), producto de los cuadernos de exploración: ya viene filtrado a viajes con origen y destino en Usaquén y con las zonas ZAT mapeadas a TAZ. La **red** (`usaquen_3d.net.xml`, con elevación) y las **zonas** (`usaquen.taz.xml`) también se generan fuera del servicio y se copian al escenario.

4.2 Diccionario de datos

4.2.1 Viaje de la encuesta (TSV de entrada)

Tabla 5: Columnas del TSV de viajes y su uso.

Campo	Tipo	Descripción y restricciones
`zat_ori`	entero	Zona de origen; se mapea a un id de TAZ. Fila inválida si falta.
`zat_des`	entero	Zona de destino. Fila inválida si falta.
`hora_ini_seg`	real	Hora de salida en segundos desde las 00:00. Solo se aceptan viajes entre `HORA_INI` (25.200) y `HORA_FIN` (36.000).
`hora_ini_hhmmss`	texto	Misma hora en formato legible (informativo).
`duracion_min`	real, opcional	Duración reportada; define el rango de duración para los presets.
`modo_principal_agrupado`	texto	Etiqueta cruda (`SurveyMode`, 11 valores) que se convierte en `TransportMode` (10 modos).
`motivo_viaje`	texto	Motivo del viaje (informativo, no usado por el pipeline).
`fexp_vj`	real	Factor de expansión. El viaje se replica `round(fexp_vj)` veces, con tope `MAX_COPIAS` = 50.

Las filas inválidas, fuera de ventana o con modo desconocido no se descartan en silencio: se cuentan en `TripGenerationStats` (`filas_invalidas`, `fuera_de_ventana`, `modos_desconocidos`) y se reportan en el resumen de la corrida.

4.2.2 Estado de una corrida (`runs/<id>/estado.json`)

Tabla 6: Campos de `estado.json` (dataclass `Corrida`).

Campo	Tipo	Descripción
`id`	texto	`AAAAMMDD-HHMMSS-<6 hex>`; clave primaria de la corrida y nombre del directorio.
`preset`	texto	Uno de `encuesta`, `mas_vehiculos`, `mas_peatones`, `dia_sin_carro`.
`conteos`	mapa modo ⟶ entero	Viajes pedidos por modo; la suma debe estar entre 1 y 100.000.
`semilla`	entero o nulo	Semilla de SUMO; nulo = la del `.sumocfg`.
`estilo`	texto	`agresivo`, `normal` o `cauteloso`.
`cierres`	lista	Cierres viales: `edge_id`, `hora_inicio_s`, `hora_fin_s`.
`estado`	texto	`en_cola` ⟶ `generando_demanda` ⟶ `ruteando` ⟶ `simulando` ⟶ `exportando` ⟶ `terminado`; o `cancelando`, `cancelada`, `error`.
`avance`	real 0–1	Avance global (según `FRANJAS_ETAPA`).

avance_etapa	real o nulo	Avance dentro de la etapa; nulo si no se puede medir.
creada, iniciada, actualizada	ISO 8601	Marcas de tiempo.
error	texto o nulo	Mensaje si falló.
resultado	objeto o nulo	viajes_generados, viajes_por_modo, trips_kepler y las estadísticas de SUMO (vehiculos_insertados, teleports, colisiones, …).

4.2.3 Trips para kepler (GeoJSON de LineString)

Tabla 7: Esquema de cada *feature* de kepler.geojson.

Campo	Tipo	Descripción
properties.id	texto	Id del vehículo o persona. Para personas, el prefijo (peaton_N, transporte_publico_N) determina el modo.
properties.mode	texto	TransportMode; el visor le asigna color.
properties.category	texto	vehiculo o persona.
properties.inicio_s, fin_s	entero	Segundos de simulación al aparecer y al salir.
properties.dist_m, avg_kmh	numérico	Distancia recorrida y velocidad media.
geometry.coordinates[i]	[lon, lat, z, t]	z relativa al mínimo del FCD; t = *unix time* = 2000-01-01 UTC + segundos de simulación (07:00 en pantalla = 25.200 s en SUMO).

4.2.4 Metadatos de una precalculada (meta.json) y volúmenes

meta.json contiene preset, conteos, semilla, generada, tamano_original, tamano_gz, resultado (el mismo resumen de una corrida) e insumos: una huella SHA-256 (16 primeros hex) de TSV, red, TAZ y .sumocfg, con los saltos de línea normalizados. Sirve para detectar que una precalculada quedó desactualizada respecto a sus insumos.

volumenes.json mapea cada edge a un contador por modo (por ejemplo {"-1012541443#0": {"bici": 12}}). Se obtiene recorriendo vehroute.xml y contando la última ruta de cada routeDistribution; reemplaza al vehroute (22 MB) en las precalculadas.

4.2.5 Intersección y cruce candidato

Tabla 8: Entidades de análisis de intersecciones.

Entidad / campo	Tipo	Descripción
Semaforo	dataclass	Un tlLogic: id, tipo, programa, offset, ciclo, fases, cruces (nodos), lon, lat, z, accesos, calles, movimientos, enlaces.
Acceso	dataclass	edge, calle, clase, carriles, velocidad_kmh, movimientos y volúmenes opcionales (volumen, volumen_hora, volumen_por_tipo).
Fase	dataclass	duracion, estado (una letra por linkIndex), luces por acceso (verde, amarillo, rojo, apagado).
ProgramaTLS	dataclass	tl_id, program_id, tipo, offset, lista de FasePrograma(duracion_s, estado); ciclo_s es la suma de duraciones.

`PuntajeCruce`	dataclass	`nodo_id`, `nodos`, `lon`, `lat`, `puntaje_total` (0–100), `nivel` (`prioritario`, `recomendado`, `observacion`), `detalle` (5 componentes en 0–1), `siniestros`, `generadores`, `tramo_m`, `volumen`, `calles`, `evidencias`, `distancia_semaforo_m`.

4.2.6 Mensaje de *frame* del simulador 3D

Cada frame del WebSocket es un objeto JSON `{tipo: "frame", t, dt, ego, manual, lados: {a: L, b: L}}`, donde cada lado `L` contiene:

- `vehiculos` y `personas`: listas de actores `[id, tipo, x, y, z, ángulo, velocidad, edge]` en coordenadas de la red.
- `luces`: `{estado, fase, restante}` del semáforo (nulo en el lado A, que no tiene).
- `metricas`: `vehiculos`, `peatones`, `en_cruce`, `detenidos_cruce`, `velocidad_media_cruce_kmh`, `espera_media_cruce_s`, `espera_lateral_s`, `colisiones`, `conflictos`, `riesgo_max`.
- `ego`: telemetría del actor seguido (velocidad, aceleración, líder, riesgo `TTC/DRAC/PET`, acción, decisiones posibles en el próximo cruce).

4.3 Estrategia de almacenamiento

Tabla 9: Ubicaciones de almacenamiento y su política.

Ubicación	Versionada	Contenido y política
`scenarios/default/`	Sí	Insumos: TSV, red 3D, TAZ, plantilla `.sumocfg`, `nombres_vias.json`, programas de semáforo guardados.
`precalculadas/<preset>/`	Sí	`trips.geojson.gz` (11,7 MB comprimido frente a 63,7 MB), `volumenes.json`, `meta.json`. El `*.geojson` sin comprimir se ignora.
`data/processed/`	Sí	Evidencia descargada una vez: siniestros SDM, POI y edificios OSM/Microsoft.
`runs/<id>/`	No	Archivos generados por corrida y por comparación; se borran al cancelar (menos `estado.json`) o al cerrar la comparación.
`cache/`	No	`semaforos.json` y `priorizacion.json`, invalidados por mtime y tamaño de la red (y por `VERSION_ANALISIS`).
`runs/<id>/output/volumenes.json`	No	Caché de volúmenes junto a la corrida, invalidada por mtime/tamaño del `vehroute`.

Decisiones de almacenamiento relevantes:

- **Archivos pesados efímeros.** El FCD de una corrida completa es de gigabytes; se escribe cada 2 s (`device.fcd.period=2`), se convierte en streaming con `iterparse` y se borra tras el export. No se genera `emission-output` (2 GB sin uso).
- **Reproducibilidad de las precalculadas.** El `.gz` se escribe con `mtime=0`, de modo que regenerar el mismo contenido da el mismo archivo y Git no registra cambios espurios.
- **Servir comprimido.** La API entrega el `.gz` sin descomprimir con `Content-Encoding: gzip` y `X-Tamano-Original`, que el visor usa para dibujar el progreso de descarga.

- **Persistencia del historial.** `estado.json` se escribe como mucho cada 2 s durante una etapa; al reiniciar el servicio, `GestorCorridas` recarga las corridas existentes.
- **Las corridas no se pisan.** Los archivos generados van a `runs/<id>/`; sin `run_id` se escribe en el escenario (comportamiento heredado para ejecutar el pipeline desde la línea de comandos).

5 Diseño de componentes (diseño detallado)

5.1 Desglose de módulos

5.1.1 Convenciones comunes del servicio

Todos los módulos de `etl/sumo/` siguen el mismo patrón, para que sean predecibles y fáciles de probar:

- Un `*Config` (`dataclass(frozen=True)`) con los parámetros y, si invoca una herramienta de SUMO, un método `to_args()`.
- Un `*Result` con lo producido y un `*Stats` mutable que se pasa y se incrementa durante el proceso (contadores de filas inválidas, viajes generados, etc.).
- Los envoltorios de subprocesos buscan el binario con `shutil.which` y lanzan una excepción propia (`*Error(RuntimeError)`).
- Los procesos largos aceptan `on_avance` (callback con el avance 0–1) y `cancelar` (`threading.Event`), y los ejecutan a través de `proceso.ejecutar`, que consulta ambos cada 0,5 s.

5.1.2 Servicio de simulación (`simulation-service/src`)

Tabla 10: Módulos de `simulation-service` y su responsabilidad.

Módulo	Responsabilidad
`api/api.py`	Aplicación FastAPI: modelos Pydantic de entrada, rutas HTTP y WebSocket, traducción de errores a códigos HTTP. Las dependencias (`obtener_gestor`, `obtener_gestor_comparaciones`) son sustituibles en pruebas con `dependency_overrides`.
`api/jobs.py`	`GestorCorridas`: cola de un trabajador, estado persistido, avance por franjas, cancelación, recuperación tras reinicio (las corridas no finales pasan a `error`, «Interrumpida: el servicio se reinició»).
`api/comparacion3d.py`	`GestorComparaciones` (máx. 2 abiertas, expiración a los 180 s sin conexión), `Comparacion` (estado y comandos en vivo), `Lado` (una instancia de SUMO con TraCI) y `correr_comparacion` (bucle en lockstep).
`etl/etl.py`	Orquesta `run_etl`, `simulate`, `export_geojson`, `export_kepler_trips`, `run_pipeline`; además `run_recorte` y `run_escenario_semaforo` para zonas.
`etl/precalculadas.py`	Genera, comprime, lista y carga los presets precalculados; calcula la huella de insumos.
`etl/sumo/configuration.py`	Única fuente de rutas y constantes: `ScenarioPaths`, `ArchivosPrecalculada`, ventana 07:00–10:00, `MAX_COPIAS` = 50, `MAX_VIAJES` = 100.000, `escribir_sumocfg_corrida`.
`etl/sumo/trips.py`	Lee el TSV, valida, expande por `fexp`, ordena por `depart` y escribe `*.trips.xml` (vehículos como `<trip>`, peatones y transporte público como `<person><walk>`).

`etl/sumo/demanda.py`	Demanda parametrizada: presets como `ReglaTransferencia`, probabilidades P(modo | duración), reparto por mayor resto.
`etl/sumo/duarouter.py`	Ejecuta `duarouter` con `--ignore-errors --repair`; el avance sale del último `depart` escrito en el `.rou.xml`.
`etl/sumo/simulation.py`	Ejecuta SUMO. Si hay `on_avance` o `cancelar`, lo maneja con TraCI (saltos de 60 s); si no, como subproceso simple.
`etl/sumo/fcd.py`, `kepler.py`	Conversión en streaming del FCD a GeoJSON de puntos (`fcd`) y a `LineString` por actor para la capa Trip (`kepler`).
`etl/sumo/vtypes.py`, `enums/`	Mapeos `SurveyMode` ⟶ `TransportMode` ⟶ `VTypeConfig` (clase de vehículo, largo, velocidad máxima, color).
`etl/sumo/estilos.py`	Estilos de conducción: `sigma`, `speedFactor`, `tau`, `lcSpeedGain` aplicados a los vTypes motorizados.
`etl/sumo/cierres.py`	Escribe un `rerouter` de SUMO para cierres viales por edge y horario.
`etl/sumo/intersecciones.py`	Catálogo de `Semaforo` desde la red (accesos, fases, enlaces), nombres de vías, volúmenes por acceso, cachés.
`etl/sumo/priorizacion.py`	Agrupación de cruces, reglas duras, evidencia, niveles y puntaje (Sección 5.2.5).
`etl/sumo/evidencia.py`	Descarga única de siniestros (SDM), POI y edificios (OSM) hacia `data/processed/`.
`etl/sumo/programas_tls.py`	Lectura, validación y escritura de programas `tlLogic`; cálculo de Webster.
`etl/sumo/subred.py`	Recorta la red (`netconvert`) y la demanda (`cutRoutes.py`) alrededor de un punto.
`etl/sumo/semaforo_nuevo.py`	Crea la variante con semáforo: parche de nodos `type="traffic_light"` con el mismo `tl` y `netconvert`.
`etl/sumo/demanda_zona.py`	Demanda de una zona: viajes reales recortados más tráfico de fondo con `randomTrips.py` por modo.
`etl/sumo/escena3d.py`	Geometría de la escena 3D: calzadas, andenes, cebras, postes, terreno IDW, edificios, rutas por acceso.
`etl/sumo/telemetria.py`	Funciones puras de riesgo (TTC, DRAC, PET), índice de riesgo y clasificación de la acción del conductor.
`etl/sumo/proceso.py`	Ejecución de subprocesos con sondeo de avance, cancelación y tiempo máximo; excepción `Cancelado`.

5.1.3 Visor web (`kepler/src`)

Tabla 11: Componentes del visor y su responsabilidad.

Componente	Responsabilidad
`App.jsx` (`Visor`)	Dueño del mapa. Descarga los trips con progreso y `AbortController`, reemplaza el dataset de kepler, activa las capas de semáforos y de priorización, y abre la ficha de intersección leyendo `visState.clicked`.
`api.js`	Cliente HTTP único (`pedir`, `descargarJson` con progreso, URLs de WebSocket). Todas las rutas son relativas a `/api`.

`hooks.js`	`useCorridas` (historial, corrida activa, consulta cada 2 s, cancelación) y `useTamanoVentana` (kepler necesita ancho y alto explícitos).
`PanelDemanda`	Presets, conteos por modo, escala, semilla y estilo de conducción; decide entre «Simular» y «Cargar simulación».
`PanelCierres`	Define cierres viales (edge y horario).
`TarjetaActividad`	Tarjeta inferior derecha con barras de progreso y botón de cancelar.
`LeyendaModos`	Leyenda de colores por modo y alternancia de la capa «Semáforos».
`FichaInterseccion`	Accesos, volúmenes, fases, edición del programa, cálculo de Webster y comparación de programas (`TablaComparacion`).
`PanelSemaforizacion`	Pestaña «Semaforizar»: elige un cruce candidato (clic en la capa o lista por puntaje), configura el tráfico de fondo y abre el simulador 3D.
`simulador3d/Simulador3D`	Ciclo de vida: crear → consultar → geometría → WebSocket; pantalla dividida «sin semáforo» y «con semáforo propuesto».
`simulador3d/escena.js`	Una escena three.js por lado: mallas estáticas fusionadas por material, actores interpolados entre frames, cámara de persecución con colisión contra edificios o aérea, conducción automática o manual por teclado.
`simulador3d/HUD`, `Minimapa`	Superposición con métricas y telemetría; minimapa OSM con maplibre (sin token).
`modos.js`	Colores por modo; el rango se arma en orden alfabético de los modos presentes porque así los asigna la escala ordinal de kepler.
`LimiteError`, `LimiteErrores`	Barreras de error (*error boundaries*) para que un fallo del visor o del 3D no tumbe la aplicación.

5.2 Lógica y algoritmos

5.2.1 Generación de viajes y orden de salida

Para cada fila válida del TSV se crea un `TripRecord` con `copies = min(round(fexp), MAX_COPIAS)`. Las copias se emiten con ids únicos y **se ordenan por `depart`** antes de escribir el XML. El orden es indispensable: el TSV no está ordenado, `duarouter` conserva el orden de entrada y SUMO **descarta** (no reordena) cualquier entidad cuya salida sea menor que la máxima ya cargada. Sin el orden se perdían cerca de 23.000 de 24.600 viajes.

5.2.2 Demanda parametrizada por presets

Un preset es una regla de transferencia modal `ReglaTransferencia(origen, destinos, duracion_mayor_a, duracion_hasta)`. Con los viajes de **todo el día** se calcula $P(m \mid r)$: la probabilidad de cada modo m en cada rango de duración r (≤ 15, 15–30, 30–60 y > 60 min), ponderada por `fexp`. Después:

1. Para cada viaje de la ventana que cumple la regla, su peso (`copies`) se reparte entre los modos destino en proporción a $P(m \mid r)$ restringida a esos destinos; los demás viajes conservan su modo.
2. Resultan *pools* por modo con pares (viaje, peso).

3. Para cada modo con conteo pedido n_m, se asignan copias a los viajes del *pool* con el método del **mayor resto** (Hamilton): $n_m \frac{w_i}{\sum_j} w_j$ se trunca y las unidades que faltan se dan a las partes decimales mayores. La suma es exacta y el resultado es determinista.
4. Si un modo no tiene *pool* en el preset (p. ej. autos en «día sin carro»), se usa el *pool* base de la encuesta.

Propiedades verificadas por pruebas: los presets conservan el total; un viaje transferido mantiene su origen/destino y su hora de salida; y con `encuesta` y sus conteos por defecto se reproduce exactamente la expansión simple de la encuesta.

5.2.3 Ejecución de SUMO bajo TraCI

`simulation._run_traci` lanza `sumo` con `--remote-port` libre, se conecta con reintentos y avanza con `simulationStep` en saltos de 60 s hasta `<end>`. Tras cada salto calcula el avance de la ventana y revisa la cancelación; ante cancelación o tiempo máximo mata el proceso. Los archivos de salida son los mismos que sin TraCI. Se usa TraCI, y no el registro de SUMO, porque con la salida redirigida a un *pipe* SUMO y `duarouter` almacenan su bitácora por bloques y esta solo llega al terminar.

5.2.4 Reducción de puntos para kepler

`kepler.reduce_points` primero colapsa cada tramo quieto (misma posición en puntos consecutivos) a su primer y último punto y luego muestrea lo que queda: siempre conserva el primer y el último punto de la entidad, y entre ellos guarda un punto cada `step` segundos (8 s en el pipeline; 5 s por defecto del módulo) o cuando el rumbo cambia al menos 20°. Las entidades que se mueven menos de `min_move` = 20 m se descartan y se cuentan. Las posiciones con `inf` o `z` fuera de ±100.000 se ignoran como inválidas.

5.2.5 Priorización de cruces sin semáforo

El análisis de la red (independiente de la corrida) y el puntaje (dependiente de los volúmenes) se separan para poder cachear la parte costosa.

Análisis (con caché `priorizacion.json`):

1. Se agrupan nodos de SUMO a menos de 20 m (extensión máxima 45 m) y se fusionan los grupos del mismo par de calles a menos de 150 m (las dos calzadas de una avenida; extensión máxima 200 m). Un cruce requiere dos calles distintas que se corten a más de 35°.
2. **Reglas duras, todas obligatorias:** participa una vía colectora (`tertiary`) o superior, sin autopistas ni troncales; la otra vía es al menos residencial; sin rampas ni glorietas a menos de 40 m; conflicto real (la vía secundaria cruza de frente o gira a la izquierda); sin semáforo a menos de 80 m.
3. **Evidencia**, calibrada contra los cruces ya semaforizados de la propia red (*backtest*): siniestros a menos de 30 m ≥ p75 (mínimo 3), o ≥ mediana con un generador cerca; tramo de la vía principal sin semáforo ≥ 600 m; volumen simulado de entrada ≥ p75. Generadores: colegios, jardines, universidades, hospitales y clínicas a menos de 200 m.

Puntaje (en cada consulta): número de evidencias «fuertes» $e \longrightarrow$ nivel `prioritario` ($e \geq 2$), `recomendado` ($e = 1$) u `observacion` (solo generador o siniestros ≥ mediana). El puntaje es un piso por nivel (75 / 50 / 25) más hasta 25 puntos:

$$\text{puntaje} = \text{piso} + 25(0,35s + 0,30v + 0,15g + 0,10t + 0,10h)$$

con s el rango percentil de siniestros, v el de volumen, $g = \min\left(\frac{g_n}{3}, 1\right)$ los generadores, $t = \min\left(\frac{\text{tramo}}{1200}, 1\right)$ y h la jerarquía. Finalmente, una *supresión de no máximos* (NMS): dos cruces del mismo nivel a menos de 120 m se estudian juntos y el de menor puntaje baja un nivel. La lógica sigue la del proyecto Semáforos de Barranquilla, sin su modelo de lenguaje.

5.2.6 Programas de semáforo y Webster

`validar_programa` exige que cada fase tenga tantos caracteres como movimientos controlados, que haya al menos una fase amarilla ≥ 3 s si hay verdes y un ciclo ≥ 15 s. El cálculo de Webster usa:

$$C_0 = \frac{1,5L + 5}{1 - Y}, \quad L = n_{\text{fases}} \cdot 3 \text{ s}, \quad Y = \sum_i \max_j \left(\frac{q_j}{s} \right)$$

con flujo de saturación s = 1.800 veh/h por carril. Los verdes se reparten en proporción a $\frac{y_i}{Y}$ (mínimo 5 s) y el ciclo mínimo es 30 s; si $Y \geq 1$ la intersección está sobresaturada y se devuelve un error. **Limitación conocida:** los accesos se asignan a las fases de forma circular (no según los movimientos compatibles reales), y si no hay volumen simulado se usa un valor de demostración de 300 veh/h. La API lo advierte en la respuesta (`nota`).

5.2.7 Variante con semáforo propuesto

`agregar_semaforo` verifica que ningún nodo del cruce tenga ya semáforo, escribe un parche `semaforo_propuesto.nod.xml` que da a **todos** los nodos del cruce `type="traffic_light"` y el mismo `tl` (un solo `tlLogic` controla el cruce completo, p. ej. las dos calzadas) y ejecuta `netconvert` con `--tls.default-type` sobre la red de la zona (`--geometry.remove false`). Luego comprueba que se generó un programa con fases.

5.2.8 Comparación 3D en lockstep

correr_comparacion(comp, cola):

1. iniciar lado A (`zona.net.xml`) y lado B (`zona_tls.net.xml`) con la misma demanda y semilla 42
2. **mientras** t < t_0 + 240 s: avanzar ambos de 20 en 20 s (calentamiento)
3. **mientras** comp.activa y t < fin:
    1. atender comandos (`manual`, `control`, `giro`, `seguir`, `inyectar`)
    2. si está en pausa: dormir 0,1 s y continuar
    3. pasos ⟵ max(1, round(velocidad)); t ⟵ t + pasos · 0,2 s
    4. **para cada lado**: avanzar a t; si manual, aplicar el mismo control al ego
    5. si t ≥ próximo_riesgo: medir riesgo y contar conflictos
    6. armar el frame (actores, luces, métricas, telemetría del ego) y publicarlo
    7. dormir lo necesario para mantener el ritmo de tiempo real
4. publicar `fin`; cerrar ambos lados

Listado 1: Pseudocódigo del bucle de la comparación 3D.

Puntos de diseño del bucle:

- **Equidad de la comparación.** Misma demanda, misma semilla y mismo conjunto de comandos aplicados a los dos lados. El ego se inyecta con el mismo id y la misma ruta en ambos (`_ruta_inyeccion`); los peatones usan `findIntermodalRoute` entre *edges* caminables.
- **Contrapresión.** La cola de frames tiene tamaño 8 y, si se llena, descarta el más antiguo: es preferible perder un frame que acumular retraso.
- **Suscripciones TraCI seguras.** Nunca se suscribe a un actor que ya salió de la red: SUMO 1.27 concatena los errores de suscripciones fallidas y falla (`Storage::writeUnsignedByte(): Invalid value`) cuando el mensaje pasa de 255 bytes. `Lado.avanzar` filtra los ids con `getIDList()`. Los vehículos inyectados que aún esperan entrar tienen `road id` vacío y se omiten.
- **Modo manual.** Acelerar y frenar se traducen a `setSpeed` (+1,8 m/s² y −3,5 m/s²), de modo que SUMO conserva sus chequeos de seguridad (no atraviesa un rojo ni choca). Los giros usan `changeTarget` hacia el último *edge* de la ruta del giro elegido.

5.2.9 Indicadores de riesgo

Funciones puras en `telemetria.py`, calculadas sobre lo que TraCI reporta del actor seguido:

- $\mathrm{TTC} = \frac{\text{gap}}{v - v_{\text{líder}}}$ si el actor se acerca al líder.
- $\mathrm{DRAC} = \frac{(v - v_{\text{líder}})^2}{2 \cdot \text{gap}}$ (m/s²).
- PET aproximado en el cruce: diferencia entre las llegadas del ego y de un vehículo en conflicto (`getJunctionFoes`) al punto de cruce.

Cada indicador se normaliza de 0 (umbral seguro) a 100 (umbral crítico o peor): TTC 6 ⟶ 1,5 s; DRAC 0 ⟶ 3,35 m/s²; PET 5 ⟶ 1,5 s. El índice de riesgo es el **mayor** de los tres (manda el más grave, que además se informa como causa) y su nivel es bajo (< 30), medio (30–59) o alto (≥ 60); los umbrales provienen de la literatura de SSM [6]. `clasificar_accion` traduce velocidad, aceleración, luz próxima, brecha y cesión de paso en una acción legible («Detenido en el semáforo», «Frenada fuerte», etc.). Las métricas por lado cuentan como conflicto un cruce con PET < 1,5 s entre vehículos a menos de 60 m del nodo.

5.2.10 Escena 3D

`geometria_escena` produce, una sola vez, el JSON que el visor convierte en geometría: calzadas, andenes, cebras y bocas de calle desde la red; un poste de semáforo por acceso; un terreno de rejilla por interpolación de distancia inversa (IDW) a partir de las alturas de calzadas y cruces; edificios de OSM y Microsoft deduplicados por solape, más un relleno ilustrativo (`relleno: true`) en los frentes de manzana vacíos que no se superponga con los reales. En el cliente, las coordenadas de SUMO se transforman como $(x, z - z_0, -(y - y_0))$ con el origen en el nodo, y las mallas estáticas se funden por material para reducir llamadas de dibujo.

6 Diseño de interfaces

6.1 Interfaces internas y externas

6.1.1 Interfaces externas

Tabla 12: Interfaces externas del sistema.

Interfaz	Medio	Uso
SUMO	Subprocesos (`sumo`, `duarouter`, `netconvert`) y TraCI por socket local	Binarios del paquete `eclipse-sumo` (1.27.1) en el `PATH` del entorno virtual; `traci==1.27.1` para el control. Las herramientas `cutRoutes.py` y `randomTrips.py` se buscan en `$SUMO_HOME/tools` y, si no, en el paquete pip `eclipse-sumo`.
Servicios de datos abiertos	HTTPS (solo al ejecutar `python -m etl.sumo.evidencia`)	SDM (ArcGIS FeatureServer, siniestros) y Overpass de OSM, con un espejo de respaldo (POI, edificios, nombres de vías). Los resultados se versionan, así que el sistema funciona sin red.
Teselas de mapa	HTTPS desde el navegador	Mapbox (base de kepler.gl, requiere `VITE_MAPBOX_TOKEN`) y OSM (minimapa del 3D, sin token).
Sistema de archivos	Lectura/escritura local	Insumos, corridas y cachés (Sección 4.3).

6.1.2 API del servicio de simulación

La API es REST sobre JSON más un WebSocket. Se sirve en `http://localhost:8000` (documentación interactiva autogenerada en `/docs`). El visor la usa a través del prefijo `/api`, que el proxy de Vite retira. Los errores usan `{"detail": ...}` con los códigos: `404` (no existe), `409` (recurso no listo o en estado incompatible), `422` (validación), `429` (límite de comparaciones abiertas).

Tabla 13: Endpoints de la API.

Método y ruta	Códigos	Descripción
`GET /health`	200	Comprobación de vida: `{"msg": "healthy"}`.
`GET /demanda/modos`	200	Modos de viaje con etiqueta, color y conteo por defecto.
`GET /demanda/presets`	200	Presets con descripción y conteos por defecto.
`GET /demanda/estilos`	200	Estilos de conducción con sus parámetros de SUMO.
`POST /simulaciones`	202, 422	Cuerpo: `{preset, conteos{modo:n}, semilla?, estilo, cierres[]}`. Encola una corrida; devuelve su estado inicial con `id`.
`GET /simulaciones`	200	Historial, la más reciente primero.
`GET /simulaciones/{id}`	200, 404	Estado, avance y resultado de una corrida.
`POST /simulaciones/{id}/cancelar`	202, 404, 409	Cancela una corrida en cola o en curso.
`GET /simulaciones/{id}/kepler-trips`	200, 404, 409	GeoJSON de trips de una corrida terminada (`FileResponse`).
`GET /precalculadas`	200	Presets ya simulados (metadatos).
`GET /precalculadas/{preset}/kepler-trips`	200, 404	Trips comprimidos (`Content-Encoding: gzip`, `X-Tamano-Original`).
`GET /simulation-output?limit=`, `GET /kepler-trips`	200, 404	Salidas de la corrida por defecto (herencia del pipeline por línea de comandos).
`GET /intersecciones?run_id=&precalculada=`	200, 404, 409	Semáforos como GeoJSON de puntos, con volumen de la corrida.
`GET /intersecciones/{tl_id}`	200, 404	Ficha: accesos, fases, volúmenes, notas sobre el origen del programa.
`GET /intersecciones/priorizacion`	200	Cruces sin semáforo por nivel y puntaje, umbrales, notas y descartes.
`GET /intersecciones/{tl_id}/programas[/{pid}]`	200, 404	Lista o detalle de programas `tlLogic`.
`PUT /intersecciones/{tl_id}/programas/{pid}`	200, 404, 422	Valida y guarda un programa editado en el escenario.

`POST /intersecciones/{tl_id}/programas/webster`	200, 422	Calcula ciclo y verdes de Webster con los volúmenes de una corrida.
`POST /intersecciones/{tl_id}/escenarios`	202, 404	Recorta red y demanda de una zona (radio 50–2.000 m).
`GET /escenarios/{run_id}`	200, 404	Estado de un recorte de zona.
`POST /intersecciones/{tl_id}/comparaciones`, `GET /comparaciones/{id}`	202, 200, 404	Compara varios programas de un semáforo por métricas de corridas de zona.
`POST /cruces/{nodo}/comparacion3d`	202, 422, 429	Cuerpo: `{lon, lat, run_id?, intensidades?, nodos?}`; intensidades 0–3.000 por hora. Prepara la comparación en segundo plano.
`GET /comparacion3d`, `GET /comparacion3d/{id}`	200, 404	Lista y estado (`preparando`, `lista`, `error`) con la etapa en curso.
`GET /comparacion3d/{id}/geometria`	200, 404, 409	Geometría de la escena; solo cuando la comparación está `lista`.
`DELETE /comparacion3d/{id}`	204, 404	Cierra la comparación y borra sus archivos.
`WS /comparacion3d/{id}`	101, cierre 1008	Transmisión en vivo (ver abajo). Se rechaza si no existe, no está lista o ya hay una ventana conectada.

Ejemplo: creación de una simulación.

```
POST /simulaciones
{"preset": "mas_peatones",
 "conteos": {"auto": 3000, "peaton": 12000, "bici": 2500, "moto": 1557},
 "semilla": 42, "estilo": "normal", "cierres": []}

202 Accepted
{"id": "20261015-101533-a1b2c3", "estado": "en_cola", "avance": 0.0, ...}
```

6.1.3 Protocolo del WebSocket de la comparación 3D

Los mensajes son JSON de texto. Una sola ventana puede estar conectada a la vez; el servidor envía un `ping` cada 5 s de inactividad.

Tabla 14: Mensajes del WebSocket `/comparacion3d/{id}`.

Sentido	**Mensaje**	**Significado**
Servidor ⟶ cliente	`{tipo:"estado", mensaje}`	Progreso del arranque y del calentamiento.
	`{tipo:"frame", t, dt, ego, manual, lados:{a, b}}`	Un paso de simulación con actores, luces, métricas y telemetría (ver Sección 4.2.6).
	`{tipo:"ego", id}`	Se inyectó un nuevo actor seguido.
	`{tipo:"giro", direccion}`	Se aceptó un giro en modo manual.
	`{tipo:"aviso", mensaje}`	Comando no aplicable (p. ej. giro imposible desde el acceso actual).

	`{tipo:"fin"}`, `{tipo:"error", mensaje}`, `{tipo:"ping"}`	Fin de la simulación, fallo (con el *stderr* de SUMO) y latido.
Cliente → servidor	`{pausa: bool}`	Pausar o reanudar.
	`{velocidad: x}`	Factor de velocidad, acotado a 0,25–20.
	`{periodo_riesgo: s}`	Cada cuántos segundos simulados se mide el riesgo (0,4–30).
	`{seguir: id}`	Cambiar el actor seguido.
	`{inyectar:{tipo, acceso, giro, estilo}}`	Insertar un actor propio con la ruta del acceso y giro elegidos.
	`{manual: bool}`, `{control: {acelerar:-1|0|1}}`, `{giro:"s"|"l"|"r"}`	Conducción manual: tomar el control, acelerar/frenar y elegir el próximo giro.

6.1.4 Interfaz interna entre capas

- **API → orquestación.** Los gestores se inyectan con `Depends`; las pruebas los reemplazan con `app.dependency_overrides` (un pipeline falso en `GestorCorridas`, una función `preparar` falsa en `GestorComparaciones`).
- **Orquestación → ETL.** `run_pipeline(scenario, sample_every, seed, demanda, run_id, exportar_puntos, on_progreso, cancelar, estilo, cierres)` es el único punto de entrada de una corrida. `on_progreso(etapa, avance | None)` y `cancelar` (`threading.Event`) cruzan todas las capas hasta el subproceso.
- **Hilo de simulación → hilo de eventos del WebSocket.** `correr_comparacion` corre en un hilo y publica en una `asyncio.Queue` mediante `call_soon_threadsafe`; los comandos del cliente entran por una `queue.Queue` (pausa, velocidad y aceleración se leen directamente del objeto `Comparacion` en cada paso).

6.2 Interfaces de usuario

La interfaz es una única pantalla a pantalla completa (`overflow: hidden`): el mapa de kepler.gl ocupa toda la ventana y los paneles flotan encima. Los colores por modo se definen en `modos.js` y se usan en el mapa, la leyenda y el panel.

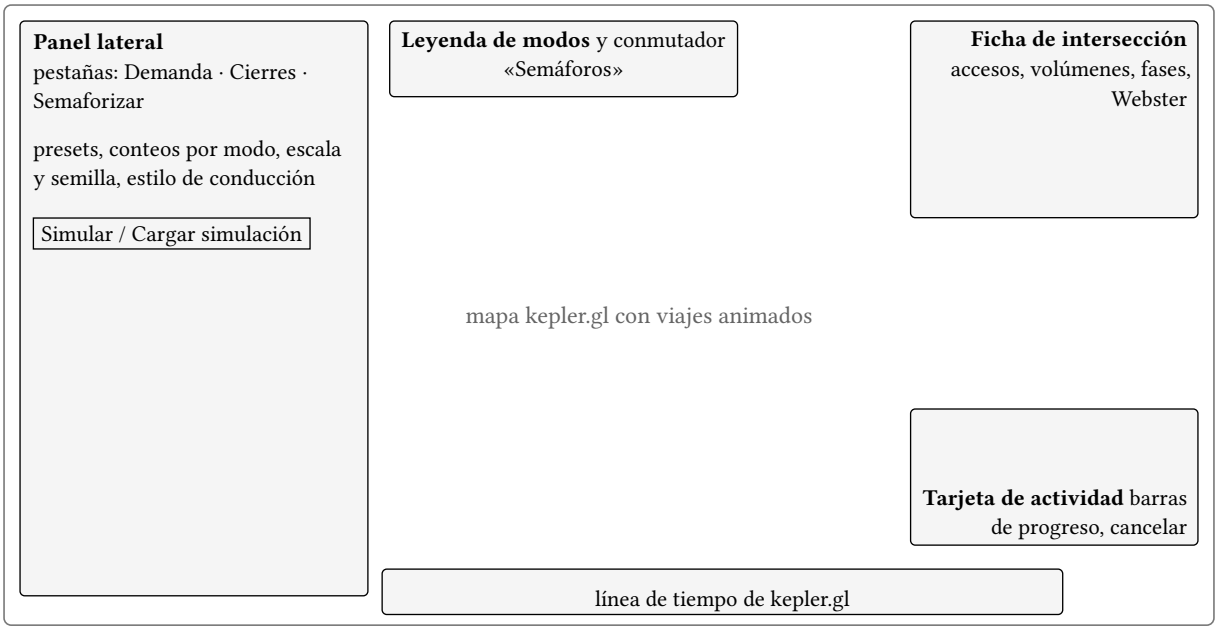


Figura 5: Esquema de la pantalla principal del visor.

Pantalla principal (Figura 5).

- **Panel de demanda:** elige un preset, ajusta viajes por modo (el total se valida contra el máximo de 100.000), semilla y estilo. Si los valores coinciden con una precalculada, el botón principal es «Cargar simulación» y la carga es inmediata; si no, «Simular» lanza una corrida.
- **Tarjeta de actividad:** muestra la etapa actual, el avance global y el de la etapa, y permite cancelar. Al terminar una corrida bien, el visor carga sus trips solo.
- **Capas de intersecciones:** «Semáforos» (puntos con la ficha al hacer clic) y «Priorización» (cruces candidatos por nivel: prioritario, recomendado, observación). Las notas de la API (por ejemplo, «sin datos de siniestros») se muestran al usuario.
- **Barra de progreso de descarga:** el visor calcula el avance con `X-Tamano-Original` de las precalculadas.

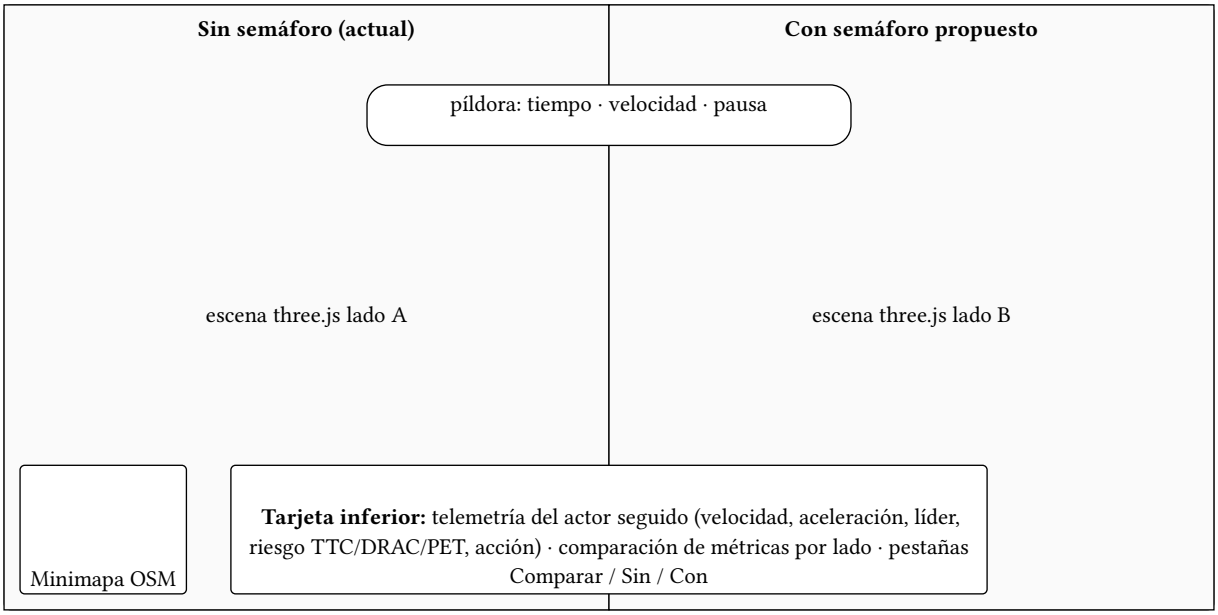


Figura 6: Esquema del simulador 3D comparativo.

Simulador 3D (Figura 6). Se abre en pantalla dividida (pestañas Comparar, Sin y Con). El HUD sigue el patrón de una píldora superior con el control de tiempo y una tarjeta inferior con la telemetría y el índice de riesgo de cada lado. El actor seguido se puede cambiar o inyectar (auto, moto, taxi, bici, peatón, entre otros) eligiendo el acceso, el giro y el estilo de conducción. En modo manual, el teclado acelera, frena y elige el giro (los comandos se aplican igual en los dos lados). La cámara puede ser de persecución, con colisión contra edificios, o aérea. El componente se carga de forma diferida (`lazy`) porque three.js y maplibre pesan, y queda envuelto en `LimiteErrores`, de modo que un fallo gráfico no tumba el mapa.

7 Atributos de calidad

7.1 Seguridad

El sistema está pensado como **herramienta local de investigación**, no como servicio expuesto a Internet. Por eso el modelo de seguridad es deliberadamente mínimo, y aquí se declara qué se cubre y qué no.

Medidas presentes:

- **Validación de entradas en la frontera.** Pydantic valida tipos y rangos: conteos 0–100.000 y total 1–100.000 viajes, semilla ≥ 0, radio de recorte 50–2.000 m, intensidades de fondo 0–3.000 por hora, factor de velocidad acotado a 0,25–20 y periodo de riesgo a 0,4–30 s. Los valores enumerados (`Preset`, `TransportMode`, `EstiloConductor`) rechazan cualquier otro texto.
- **Sin ejecución de comandos arbitrarios.** Los subprocesos se lanzan con una lista de argumentos (sin *shell*) y los parámetros que llegan del cliente nunca forman parte de la línea de comandos sin pasar por un tipo validado; los ids de nodo que llegan del cliente se escapan antes de escribirse en el XML del parche de semáforo y solo se usan si existen en la red recortada.
- **Recursos acotados.** Un trabajador para las corridas, máximo dos comparaciones 3D, una sola ventana por comparación y expiración por inactividad evitan que un cliente agote la máquina.
- **Secretos fuera del código.** El token de Mapbox se lee de `kepler/.env` (ignorado por Git; la plantilla es `.env.example`).
- **Manejo de errores.** Una corrida fallida no tumba el servicio: se registra el error (últimos 2.000 caracteres) en su `estado.json`.

Limitaciones aceptadas (fuera de alcance de la versión actual):

- **No hay autenticación ni autorización**, no hay roles ni cifrado en tránsito (HTTP local) y no se configura CORS: el visor accede por el proxy de Vite del mismo origen. Si el servicio se publicara, habría que añadir autenticación (p. ej. OAuth 2.0/JWT), TLS y limitación de tasa.
- Algunas rutas (`/escenarios/{run_id}`) leen `runs/<run_id>/estado.json` directamente a partir del identificador recibido; conviene validar el formato del id (`[0-9A-Za-z-]+`) antes de exponer el servicio fuera de la máquina local.
- Los datos son abiertos o agregados (encuesta expandida, siniestros públicos): no contienen información personal identificable.

7.2 Rendimiento y escalabilidad

Tiempos medidos de referencia. Una corrida completa de la encuesta (24.600 viajes) tarda unos 12 min: generación de demanda y `duarouter` ≈ 1,5 min, SUMO ≈ 8 min y exportación ≈ 2 min. De ahí salen las franjas de avance (`FRANJAS_ETAPA`). La comparación 3D tarda en preparar la zona decenas de segundos, más 240 s simulados de calentamiento al conectarse.

Tabla 15: Estrategias de rendimiento.

Problema	Estrategia
SUMO es lento	Presets precalculados y versionados: abrir el visor muestra una simulación completa sin esperar. Corridas nuevas en segundo plano con cancelación.
Salida enorme (FCD 1,4 GB)	`device.fcd.period=2`, sin `emission-output`, lectura en streaming con `iterparse`, borrado del FCD tras exportar.
GeoJSON de 64 MB en el navegador	Reducción de puntos por tiempo y rumbo, descarte de entidades casi inmóviles, compresión gzip (11,7 MB) servida sin recomprimir, barra de progreso.
Análisis de red costoso	Cachés en disco (`semaforos.json`, `priorizacion.json`, `volumenes.json`) invalidadas por mtime y tamaño; las consultas dependientes de la corrida son baratas.
3D en tiempo real	Paso de 0,2 s, cola de frames de tamaño 8 con descarte del más antiguo, ritmo de tiempo real con factor de velocidad, mallas fusionadas por material, interpolación entre frames y carga diferida de three.js.
Concurrencia	Un `ThreadPoolExecutor(1)` para corridas; hasta 2 comparaciones 3D (cada una son 2 procesos SUMO).

Escalabilidad. El diseño escala en **volumen de demanda** (hasta 100.000 viajes por corrida) y en **número de escenarios** (cada uno en `scenarios/<nombre>/`), pero no en **número de usuarios simultáneos**: la cola de un solo trabajador y el estado en memoria y disco local suponen una instancia. Para soportar varios usuarios habría que mover el estado de las corridas a una base de datos o cola externa y ejecutar trabajadores en procesos distintos (Sección 8.2, ADR-02).

Recuperación (*failover*). No hay alta disponibilidad. Lo que sí existe es recuperación ante reinicio: el historial se reconstruye desde `runs/*/estado.json` y las corridas que estaban activas se marcan como `error` («Interrumpida: el servicio se reinició»). Las comparaciones 3D, en cambio, son efímeras y se pierden.

7.3 Mantenibilidad y pruebas

- **Pruebas automáticas:** 103 pruebas con `pytest` en `simulation-service/test/` (demanda y presets, viajes, FCD, kepler, intersecciones, priorización, progreso y cancelación, semáforo nuevo, escena 3D, telemetría, precalculadas, API y comparación 3D). Las dependencias externas se aíslan con `dependency_overrides` y datos mínimos, sin ejecutar SUMO completo.
- **Riesgo conocido:** el flujo de CI (`.github/workflows/tests.yml`) instala solo el `requirements.txt` de la raíz y corre `tests/`, que es un marcador. **Las pruebas de `simulation-service/` no se ejecutan en CI**; deben correrse a mano (`pytest -v` dentro de `simulation-service/`). El visor no tiene pruebas automáticas.
- **Convenciones:** PEP 8, nombres y comentarios en español, patrón `Config/Result/Stats` uniforme, y una sola fuente de verdad para rutas (`configuration.py`).
- **Sincronización con el SRS:** cada requisito tiene un estado (Implementado/Parcial/Pendiente) que debe actualizarse al construir la funcionalidad.

7.4 Reproducibilidad y portabilidad

- Versiones fijadas (`eclipse-sumo==1.27.1`, `traci==1.27.1`, `fastapi==0.136.3`, `pyproj==3.8.0`), semilla configurable, ordenamiento determinista de viajes y huella de insumos en las precalculadas.

- El mismo código corre en Linux y Windows. Se contempla el bloqueo de archivos de Windows al borrar el directorio de una comparación (hasta 5 reintentos en un hilo aparte) y se normalizan los saltos de línea al calcular la huella de insumos.

7.5 Observabilidad y usabilidad

- **Observabilidad:** estado y avance de cada corrida consultables por la API; el *stderr* de SUMO se anexa al error de una corrida o comparación fallida; las estadísticas (`teleports`, colisiones, viajes descartados) se incluyen en el resultado.
- **Honestidad del modelo:** la API acompaña los datos con notas cuando una limitación afecta su lectura (programa de OSM, ausencia de volúmenes, falta de datos de siniestros).
- **Usabilidad:** progreso visible, cancelación, valores por defecto coherentes con la encuesta, mensajes de error en español y un visor en una sola pantalla.

8 Apéndices y referencias

8.1 Pila tecnológica (Tech Stack)

Tabla 16: Pila tecnológica.

Capa	Tecnología	Uso
Simulación	SUMO 1.27.1 (`eclipse-sumo`), `traci` 1.27.1, `sumolib`	Simulación microscópica, ruteo (`duarouter`), construcción de red (`netconvert`), control en vivo (TraCI).
Servicio	Python ≥ 3.11, FastAPI 0.136.3 (`fastapi[standard]`), Pydantic, `uvicorn`	API REST/WebSocket, validación.
Geodesia	`pyproj` 3.8.0	Coordenadas UTM de la red a lon/lat.
Pruebas	`pytest` 9.1.1; `matplotlib` (validación)	Pruebas y gráficos de validación (`validation/`).
Visor	React 18.3, Vite 6, `vite-plugin-node-polyfills`	Aplicación de una página con proxy `/api`.
Mapa	kepler.gl 3.1 (`@kepler.gl/*`), Redux 4, `react-palm`, `styled-components` 6.1.8, `react-intl`	Mapa y capa Trip.
3D	three.js 0.169, maplibre-gl 4.7.1	Escena comparativa y minimapa OSM.
Calidad	ESLint 10 (visor), pylint (CI, sin bloqueo)	Análisis estático.
Documentación	Typst (`.typ` ⟶ PDF), IEEE 1016, 830, 1058	SRS, SPMP, SDD, VFP.
Datos	Encuesta de Movilidad 2023 (TSV), OSM, siniestros SDM, edificios OSM y Microsoft	Insumos versionados en `data/processed/` y `scenarios/`.
Control de versiones y CI	Git, GitHub Actions	Flujo `tests.yml` (ver Sección 8.4).

Puesta en marcha.

```
# Servicio (desde simulation-service/)
python3.11 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
```

```
python -m etl.etl                 # pipeline completo (varios minutos)
python -m etl.precalculadas       # regenerar presets precalculados
uvicorn api.api:app --reload      # API en :8000, documentación en /docs
pytest -v

# Visor (desde kepler/)
npm install && npm run dev        # requiere la API en :8000 y VITE_MAPBOX_TOKEN
en .env
```

El SDD se compila con `typst compile docs/sdd/sdd.typ` (desde la raíz del repositorio puede usarse `--root docs/sdd`). Los diagramas usan el paquete `fletcher` de Typst, que se descarga en la primera compilación.

8.2 Registros de decisiones (ADRs)

Tabla 17: Registro de decisiones de arquitectura.

ID	Decisión	Motivo	Alternativas y consecuencias
ADR-01	Servicio único FastAPI con módulos internos (monolito modular).	Una sola máquina; SUMO domina el costo; simplifica cancelación, rutas de archivos y pruebas.	Microservicios: más infraestructura sin beneficio. Costo: no escala en usuarios.
ADR-02	Un trabajador (`ThreadPoolExecutor(1)`) y cola en memoria para las corridas.	SUMO consume CPU y memoria; dos corridas a la vez se estorban y no terminan antes.	Celery/Redis: sobredimensionado. Costo: una sola instancia; las corridas activas se pierden si el servicio se reinicia (se marcan `error`).
ADR-03	Archivos en lugar de base de datos.	Los resultados de SUMO ya son archivos; un escenario; cero operación.	PostgreSQL/PostGIS: útil con muchos escenarios y usuarios. Costo: consultas ad hoc, sin transacciones.
ADR-04	Orquestar SUMO como subprocesos y TraCI.	Binarios probados; TraCI permite medir avance, cancelar y controlar actores.	Bindings C++/libsumo: más rápido, más acoplado. Costo: latencia de socket en el 3D.
ADR-05	Ordenar los viajes por salida antes de rutear.	SUMO descarta lo que llegue desordenado (23.000 de 24.600 viajes se perdían).	Ordenar en `duarouter`: no lo hace. Costo: pasar todos los viajes por memoria (aceptable).
ADR-06	Presets como reglas de transferencia modal con P(modo | duración) y mayor resto.	Conservan el total y los orígenes/destinos y son deterministas; con `encuesta` reproducen la encuesta.	Reponderar al azar: menos reproducible. Costo: el reparto depende de la duración reportada.
ADR-07	Precalcular y versionar los cuatro presets.	Una corrida tarda 12 min; el visor debe mostrar algo útil al abrir.	Generar al arrancar: lento. Costo: 12 MB por preset en Git y la huella de insumos para detectarlos obsoletos.

ADR-08	Usar kepler.gl (capa Trip) para la visualización principal.	Animación de trayectorias lista, filtros y línea de tiempo.	deck.gl a mano: más control, más trabajo. Costo: dependencia pesada, Redux y orden alfabético de colores.
ADR-09	Simulador 3D comparativo con dos SUMO en lockstep, en lugar de una vista en vivo de un semáforo existente.	La decisión de interés es «¿qué cambia si pongo un semáforo aquí?», que exige misma demanda y semilla en ambos lados.	Comparar métricas finales de dos corridas: menos intuitivo. Costo: 2 procesos SUMO por comparación y un límite de 2 comparaciones.
ADR-10	Priorización con reglas y evidencia calibrada contra los semáforos existentes, sin modelo de lenguaje.	Explicable, reproducible y sin dependencia externa; se adapta el enfoque de Barranquilla.	Puntaje opaco con LLM. Costo: umbrales fijados a mano y a la red de Usaquén.
ADR-11	Transporte público como `<person><walk>`.	No hay GTFS ni paradas; evita inventar un servicio.	Buses con `gtfs2pt` y `personTrip modes="public"`: pendiente. Costo: subestima la carga sobre las vías.
ADR-12	Escena 3D con relleno ilustrativo marcado (`relleno: true`).	OSM y Microsoft no cubren todas las manzanas; una escena vacía no se entiende.	Solo edificios reales: escena vacía. Costo: parte de la escena no es real.
ADR-13	Documentación en Typst y código en español.	Uniformidad con el SRS/ SPMP y con el equipo y jurado.	LaTeX/Word: menos mantenible. Costo: ecosistema más pequeño.

8.3 Trazabilidad con el SRS

Tabla 18: Trazabilidad entre grupos de requisitos y componentes de diseño.

Grupo del SRS	**Cubre**	**Componentes del diseño**
F1, F2, F3	Ingestión de datos, matrices OD, red y TAZ.	`trips.py`, `enums/`; notebooks `F1.x`/ `F2.x` (fuera del servicio); red y TAZ copiadas a `scenarios/`.
F4	Archivos de simulación.	`etl.py`, `duarouter.py`, `simulation.py`, `configuration.py`.
F5	Variación paramétrica.	`demanda.py`, `estilos.py`, `cierres.py`, `jobs.py`, `precalculadas.py`.
F6	Módulo web.	`kepler/` completo; `GET/POST /simulaciones`, `/precalculadas`.
F7	Validación y reportes.	`fcd.py`, `kepler.py`, estadísticas en `resumen_corrida`; `validation/` (histograma de salidas).
F8	Intersecciones y semaforización.	`intersecciones.py`, `priorizacion.py`, `evidencia.py`, `programas_tls.py`, `subred.py`,

		`FichaInterseccion`, `PanelSemaforizacion`.
F9	Simulador 3D comparativo.	`comparacion3d.py`, `semaforo_nuevo.py`, `demanda_zona.py`, `escena3d.py`, `telemetria.py`, `simulador3d/`.

8.4 Limitaciones conocidas del diseño

- **Transporte público** simulado como caminata; sin buses, paradas ni GTFS.
- **Programas de semáforo** generados por `netconvert` desde OSM; no son los reales de la Secretaría de Movilidad.
- **Webster simplificado:** reparto circular de accesos a fases y valor de demostración cuando no hay volumen.
- **Demanda sintética en la comparación 3D:** los viajes de la encuesta son escasos en una zona de 350 m, así que se completan con tráfico de fondo por modo; los resultados comparan escenarios entre sí, no predicen un volumen real.
- **Una instancia:** estado local, cola de un trabajador, sin autenticación.
- **CI incompleto:** las pruebas de `simulation-service` y el visor no se ejecutan en CI.
- **Sin reanudación por fase:** una corrida interrumpida debe repetirse completa.
- **Rutas heredadas:** el README raíz menciona `src/pipeline/` y `src/simulation/`, que no existen; todo el código del pipeline está en `simulation-service/src/`.

9 Referencias

[1] IEEE, «IEEE Std 1016-2009 – IEEE Standard for Information Technology – Systems Design – Software Design Descriptions», 2009.

[2] IEEE, «IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications», 1998.

[3] IEEE, «IEEE Std 1058-1998 – IEEE Standard for Software Project Management Plans», 1998.

[4] P. A. Lopez, M. Behrisch, L. Bieker-Walz, y J. Erdmann, «Microscopic Traffic Simulation using SUMO», en *21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575-2582.

[5] F. V. Webster, «Traffic Signal Settings (Road Research Technical Paper No. 39)», Her Majesty's Stationery Office, 1958.

[6] D. Gettman, L. Pu, T. Sayed, y S. Shelby, «Surrogate Safety Assessment Model and Validation (FHWA-HRT-08-051)», Federal Highway Administration, 2008.

[7] Secretaría Distrital de Movilidad de Bogotá, «Encuesta de Movilidad de Bogotá 2023», Alcaldía Mayor de Bogotá, 2023.

[8] ISO/IEC, «ISO/IEC 12207:2008 – Systems and software engineering – Software life cycle processes», 2008.

[9] Object Management Group, «Business Process Model and Notation (BPMN) Version 2.0», 2011.

[10] J. de D. Ortúzar y L. G. Willumsen, *Modelling Transport*, 4.ª ed. Wiley, 2011.

[11] R. Wirth y J. Hipp, «CRISP-DM: Towards a standard process model for data mining», 2000.

[12] K. Schwaber y J. Sutherland, *The Scrum Guide*. 2020.

[13] ISO/IEC/IEEE, «ISO/IEC/IEEE 29119-1:2022 – Software and systems engineering – Software testing», 2022.